Problem Statement

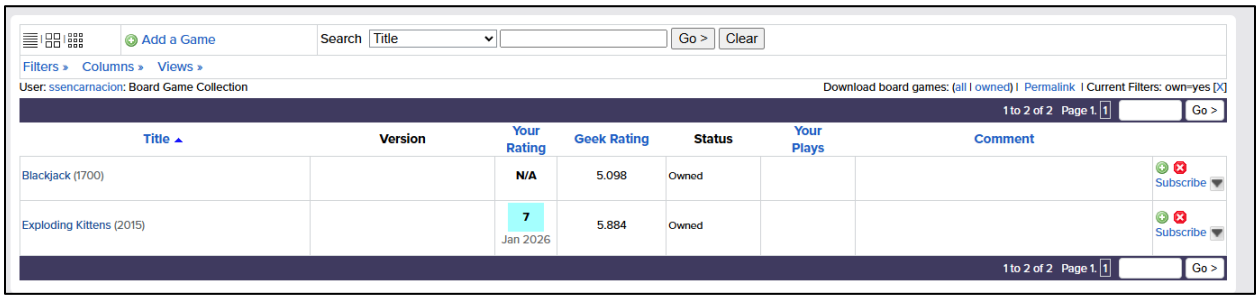
boardgamegeek.com is a popular resource that board game fans use to make informed decisions before buying games. However, reviews are highly subjective, and finding games that you like is easier when you follow people with similar tastes.

Create an interface that takes two Board Game Geek usernames as input and returns some sort of similarity score as output. The similarity score should be reflective of how similar their tastes are. You can limit your dataset to the top 100 games based on the Geek Rating, but put in the assumptions when you do so.

A sample data folder will be utilized that contains XML files. Your application should be designed to ingest XML files from a local `/data` directory by default. The sample data folder represents a raw "Data Lake." Try to have it include a mix of clean and dirty data (including **malformed XML nodes**, **unescaped characters**, and **type mismatches**).

1.0 Getting Started

To start the problem, I've created an account in the platform, and added some games along with my ratings for them.



1.1 BGG XML API2

According to the [official documentation](), the /collection endpoint can be queried with various parameters. Since it is stated that **"**finding games that you like is easier when you follow people with similar tastes," we are interested with obtaining the list of ratings of the users. The parameters **username=NAME** and **stats=1** can be utilized to retrieve a user's rated collection items and their corresponding ratings.

From [https://boardgamegeek.com/xmlapi2/collection?username=ssencarnacion&stats=1](), we are actually able to obtain information about my rating(s), as shown below (image is cropped).


```
This XML file does not appear to have any style information associated with it. The document tree is shown below.

▼<items totalitems="2" termsofuse="https://boardgamegeek.com/xmlapi/termsofuse" pubdate="Thu, 15 Jan 2026 11:57:01 +0000">
  ▼<item objecttype="thing" objectid="10816" subtype="boardgame" collid="141301052">
     <name sortindex="1">Blackjack</name>
     <yearpublished>1700</yearpublished>
     <image>https://cf.geekdo-images.com/hDixWez4o3x9D0f4pVl5bA__original/img/dB051FZ06gGOag8V90J8mtOkBNI=/0x0/filters:format(jpeg)/pic163405.jpg</image>
     <thumbnail>https://cf.geekdo-images.com/hDixWez4o3x9D0f4pVl5bA__small/img/qsaFz6U5aPwkUx7dPosoGWags9w=/fit-in/200x150/filters:strip_icc()/pic163405.jpg</thumbnail>
   ▼<stats minplayers="2" maxplayers="10" minplaytime="30" maxplaytime="30" playingtime="30" numowned="905">
     ▼<rating value="N/A">
        <usersrated value="2222"/>
        <average value="4.79212"/>
        <bayesaverage value="5.09777"/>
        <stddev value="1.84745"/>
        <median value="0"/>
      ▼<ranks>
         <rank type="subtype" id="1" name="boardgame" friendlyname="Board Game Rank" value="29888" bayesaverage="5.09777"/>
         <rank type="family" id="4666" name="abstracts" friendlyname="Abstract Game Rank" value="1488" bayesaverage="4.71204"/>
         <rank type="family" id="5498" name="partygames" friendlyname="Party Game Rank" value="960" bayesaverage="4.81684"/>
       </ranks>
      </rating>
    </stats>
    <status own="1" prevowned="0" fortrade="0" want="0" wanttoplay="0" wanttobuy="0" wishlist="0" preordered="0" lastmodified="2026-01-15 02:25:51"/>
    <numplays>0</numplays>
  </item>
▼<item objecttype="thing" objectid="172225" subtype="boardgame" collid="141300525">
    <name sortindex="1">Exploding Kittens</name>
    <yearpublished>2015</yearpublished>
    <image>https://cf.geekdo-images.com/N8bL53-pRU7zaXDTrEaYrw__original/img/0ciN1VZYifUd0qIDO0e8cGXmiss=/0x0/filters:format(png)/pic2691976.png</image>
    <thumbnail>https://cf.geekdo-images.com/N8bL53-pRU7zaXDTrEaYrw__small/img/3tH4pIc1UdzkdOtXc6MgVQ59BC0=/fit-in/200x150/filters:strip_icc()/pic2691976.png</thumbnail>
  ▼<stats minplayers="2" maxplayers="5" minplaytime="15" maxplaytime="15" playingtime="15" numowned="90291">
    ▼<rating value="7">
        <usersrated value="40502"/>
        <average value="6.06891"/>
        <bayesaverage value="5.8838"/>
        <stddev value="1.7452"/>
        <median value="0"/>
      ▼<ranks>
         <rank type="subtype" id="1" name="boardgame" friendlyname="Board Game Rank" value="3933" bayesaverage="5.8838"/>
         <rank type="family" id="5498" name="partygames" friendlyname="Party Game Rank" value="576" bayesaverage="5.68031"/>
       </ranks>
      </rating>
```


1.2 Fishing in the Data Lake

The very first step to solving the problem at hand is to actually try and get information from the XML file. The relevant information we need to obtain are a username, the games in their collection, and its corresponding ratings. Once we are able to extract this information, we can almost solve the problem as a whole.

But what files should we choose? Given that there is so many residing in the data lake?

collection@stats=1&subtype=boardgam...	14 Jan 2026 1:42 am	XML File	48 KB
collection@stats=1&subtype=boardgam...	14 Jan 2026 1:42 am	XML File	48 KB
collection@stats=1&subtype=boardgam...	14 Jan 2026 1:50 am	XML File	2 KB
collection@stats=1&subtype=boardgam...	14 Jan 2026 1:59 am	XML File	1 KB
collection@stats=1&subtype=boardgam...	14 Jan 2026 1:42 am	XML File	1 KB
geeklist_25-1	14 Jan 2026 1:42 am	Chrome HTML Do...	1 KB
geeklist_251	14 Jan 2026 1:42 am	File	4 KB
geeklist_251@comments=1	14 Jan 2026 1:42 am	File	18 KB
guild@id=0&members=1	14 Jan 2026 1:42 am	XML File	1 KB
guild@id=930&members=1&page=2	14 Jan 2026 1:42 am	XML File	2 KB
guild@id=930&members=1	14 Jan 2026 1:42 am	XML File	3 KB

Our hint lies in the documentation! Not every file here will be relevant, as **only those originating from the collection API will be needed**!

ASSUMPTION # 1: Every file in the data lake must be following the existing naming convention from the current files. The relevant files we will be looking for must have its filename starting with: "collection@stats=1&subtype=boardgame&username=<username>"

2.0 Parsing XML Information

To get the necessary information from the XML file, I have created a simple program to do just that. The first 2 lines of the parser function allows us to obtain the username from the filename. Afterwards, the function parses the XML file, finds all game items, and for each game retrieves its name and rating (converting it to a number when possible) before printing the username, total number of games, and each game with its rating.

parser_v1.py U collection@stats=1&subtype=boardgame&username=ssencarnacion&version=0.xml U

parser_v1.py > parser

```python
import xml.etree.ElementTree as ET

def parser(filepath):
    # extract username from filename
    filename = filepath.split("/")[-1].split("\\")[-1]
    username = filename.split("username=", 1)[1].split("&")[0]

    # parse XML and extract games and their respective ratings
    # https://docs.python.org/3/library/xml.etree.elementtree.html
    items = ET.parse(filepath).getroot().findall("item")
    print(f"username: {username}")
    print(f"total games: {len(items)}")

    for item in items:
        name = item.findtext("name")
        rating_value = item.find("stats/rating").get("value")

        try:
            rating_value = float(rating_value)
        except (TypeError, ValueError):
            rating_value = "N/A"
        print(f"{name} : {rating_value}")

username = "ssencarnacion"
xml_file = f"data/collection@stats=1&subtype=boardgame&username={username}&version=0.xml"
parser(xml_file)
```

As we have seen previously, I only have 2 games in my collection. The parser function was able to successfully obtain the games in my collection, along with their corresponding ratings.

```
username: ssencarnacion97-988ab0c799f8
total games: 2
Blackjack : N/A
Exploding Kittens : 7.0
```

The parser function works, but it requires the XML file to be well-formed and free of errors.

2.1 Improving the Parser

Fortunately, for our case, we can iteratively improve our parser function by utilizing the existing files from our sample data lake. With each iteration, we can augment our parser function to be more robust against different inputs.

a. Different Versions? Expansions?

According to the official documentation, the **version** parameter gives us more information regarding a certain version of a boardgame. For user fagentu007, we can see that there is no difference in the output when version is set to 0 or 1. This is expected, since we are not interacting / do not need to interact with the <version> tag.

```
username: fagentu007
total games: 40
The Ancient World : N/A
Auf den Spuren von Marco Polo : 9.0
Biblios : 6.0
Bora Bora : N/A
Camel Up : 6.0
The Castles of Burgundy : N/A
Catan : 3.0
Caverna: The Cave Farmers : 9.5
Chess : 4.0
Dead Men Tell No Tales : N/A
Eclipse : N/A
Eldritch Horror : 9.5
Imperial Settlers : 7.0
Istanbul : 7.0
Jaipur : 6.0
Jenga : 6.0
Last Will : N/A
Legacy: Gears of Time : 7.0
Lewis & Clark : N/A
Lords of Waterdeep : 8.0
Lords of Waterdeep: Scoundrels of Skullport : 8.0
Love Letter : 7.0
Mage Knight Board Game : 7.0
Mysterium : N/A
Orléans : N/A
Pandemic : 8.8
Pandemic: On the Brink : 8.0
Power Grid Deluxe: Europe/North America : 8.0
Praetor : 5.0
Race for the Galaxy : 8.1
Robinson Crusoe: Adventures on the Cursed Island : 9.5
Russian Railroads : 9.0
Saboteur : 8.0
Saboteur 2 (expansion-only editions) : 7.0
Scrabble : 4.0
Star Trek: Frontiers : N/A
Thunderstone Advance: Towers of Ruin : 9.0
Trajan : N/A
Tzolk'in: The Mayan Calendar : 8.0
Tzolk'in: The Mayan Calendar – Tribes & Prophecies : 8.0
```

```
username: fagentu007
total games: 40
The Ancient World : N/A
Auf den Spuren von Marco Polo : 9.0
Biblios : 6.0
Bora Bora : N/A
Camel Up : 6.0
The Castles of Burgundy : N/A
Catan : 3.0
Caverna: The Cave Farmers : 9.5
Chess : 4.0
Dead Men Tell No Tales : N/A
Eclipse : N/A
Eldritch Horror : 9.5
Imperial Settlers : 7.0
Istanbul : 7.0
Jaipur : 6.0
Jenga : 6.0
Last Will : N/A
Legacy: Gears of Time : 7.0
Lewis & Clark : N/A
Lords of Waterdeep : 8.0
Lords of Waterdeep: Scoundrels of Skullport : 8.0
Love Letter : 7.0
Mage Knight Board Game : 7.0
Mysterium : N/A
Orléans : N/A
Pandemic : 8.8
Pandemic: On the Brink : 8.0
Power Grid Deluxe: Europe/North America : 8.0
Praetor : 5.0
Race for the Galaxy : 8.1
Robinson Crusoe: Adventures on the Cursed Island : 9.5
Russian Railroads : 9.0
Saboteur : 8.0
Saboteur 2 (expansion-only editions) : 7.0
Scrabble : 4.0
Star Trek: Frontiers : N/A
Thunderstone Advance: Towers of Ruin : 9.0
Trajan : N/A
Tzolk'in: The Mayan Calendar : 8.0
Tzolk'in: The Mayan Calendar – Tribes & Prophecies : 8.0
```

Note that there are similar games, as in the case of Pandemic and Pandemic: On the Brink. According to the official documentation, “the default (or using subtype=boardgame) returns both boardgame and boardgameexpansion's in your collection... but incorrectly gives subtype=boardgame for the expansions.”

As we can see, Pandemic and Pandemic: On the Brink have **distinct objectids**, even though the latter is an expansion of the former.


```
▼<item objecttype="thing" objectid="30549" subtype="boardgame" collid="29012664">
   <name sortindex="1">Pandemic</name>
   <yearpublished>2008</yearpublished>
   <image>https://cf.geekdo-images.com/S3ybV1LAp-8SnHIXLLjVqA__original/img/IsrvRLpUV1TEyZsO5rC-btXaPz0=/0x0/filters:format(jpeg)/pic1
   <thumbnail>https://cf.geekdo-images.com/S3ybV1LAp-8SnHIXLLjVqA__small/img/oqViRj6nVxK3m36NluTxU1PZkrk=/fit-in/200x150/filters:strip_
  ▼<stats minplayers="2" maxplayers="4" minplaytime="45" maxplaytime="45" playingtime="45" numowned="222314">
    ▼<rating value="8.8">
       <usersrated value="133614"/>
       <average value="7.51685"/>
       <bayesaverage value="7.40765"/>
       <stddev value="1.3386"/>
       <median value="0"/>
     </rating>
   </stats>
   <status own="1" prevowned="0" fortrade="0" want="0" wanttoplay="0" wanttobuy="0" wishlist="0" preordered="0" lastmodified="2015-06-
   <numplays>11</numplays>
 </item>
▼<item objecttype="thing" objectid="40849" subtype="boardgame" collid="31261959">
   <name sortindex="1">Pandemic: On the Brink</name>
   <yearpublished>2009</yearpublished>
   <image>https://cf.geekdo-images.com/MOsfIJrEqkGMUlo09TL3rw__original/img/cJ2kOhpNeVakyUn4SfzD8kwO6n8=/0x0/filters:format(jpeg)/pic7
   <thumbnail>https://cf.geekdo-images.com/MOsfIJrEqkGMUlo09TL3rw__small/img/CSEB6i26tqktFOXyh2IgqtR8zpE=/fit-in/200x150/filters:strip_
  ▼<stats minplayers="2" maxplayers="5" minplaytime="45" maxplaytime="45" playingtime="45" numowned="53515">
    ▼<rating value="8">
       <usersrated value="18311"/>
       <average value="8.02984"/>
       <bayesaverage value="7.69655"/>
       <stddev value="1.1211"/>
       <median value="0"/>
     </rating>
   </stats>
```


A possible workaround here is to have a dictionary that stores an expansion's objectid and the original game's objectid as a key-value pair. Should we decide to treat a game and its expansion as the same, then we can simply refer to that dictionary.

ASSUMPTION # 2: For comparison purposes, **we will be considering different expansions of the game as different games.**

But what about different versions of the game? To know what actually is the difference, I've modified my collection to contain these two games.

User: ssencarnacion: Board Game Collection — Download board games: (all | owned) | Permalink | Current Filters: own=yes [X]

1 to 2 of 2 Page 1. 1 Go >

Title	Version	Your Rating	Geek Rating	Status	Your Plays	Comment	
CATAN (English edition 2025) (2025)	English edition 2025 Year: 2025	N/A	6.905	Owned			Subscribe
卡坦島 (2025)	Chinese edition Year: 2025	8 Jan 2026	6.905	Owned			Subscribe

The corresponding XML file looks like the following. Observe that even though these 2 games are different by name, they are still the **same game**, as seen in the object id. They are just different versions of the same game.


```
▼<items totalitems="2" termsofuse="https://boardgamegeek.com/xmlapi/termsofuse" pubdate="Thu, 15 Jan 2026 14:34:17 +0000">
  ▼<item objecttype="thing" objectid="13" subtype="boardgame" collid="141308664">
     <name sortindex="1">CATAN</name>
     <yearpublished>2025</yearpublished>
     <image>https://cf.geekdo-images.com/0XODRpRe1ZBFUffEcqT5-Q__original/img/oRc0AomWA9ZtFqQDZiZbIyKE1j0=/0x0/filters:format(png)/pic9156909.png</image>
     <thumbnail>https://cf.geekdo-images.com/0XODRpRe1ZBFUffEcqT5-Q__small/img/SNVfF23OQafv3u8xdFolJnMkBoM=/fit-in/200x150/filters:strip_icc()/pic9156909.png</thumbnail>
    ▶<stats minplayers="3" maxplayers="4" minplaytime="60" maxplaytime="120" playingtime="120" numowned="234608">
     ...
     </stats>
     <status own="1" prevowned="0" fortrade="0" want="0" wanttoplay="0" wanttobuy="0" wishlist="0" preordered="0" lastmodified="2026-01-15 07:30:03"/>
     <numplays>0</numplays>
   </item>
  ▼<item objecttype="thing" objectid="13" subtype="boardgame" collid="141308666">
     <name sortindex="1">卡坦島</name>
     <originalname>CATAN</originalname>
     <yearpublished>2025</yearpublished>
     <image>https://cf.geekdo-images.com/OAEm41W-ZMMP59L6_2bQ5Q__original/img/vIC1hopizr5bMIuAK8N9xcXlONY=/0x0/filters:format(jpeg)/pic9159960.jpg</image>
     <thumbnail>https://cf.geekdo-images.com/OAEm41W-ZMMP59L6_2bQ5Q__small/img/2_Ep-kJq3utrq3TN2FiwuKsRwQs=/fit-in/200x150/filters:strip_icc()/pic9159960.jpg</thumbnail>
    ▶<stats minplayers="3" maxplayers="4" minplaytime="60" maxplaytime="120" playingtime="120" numowned="234608">
```

Running parser_v1 on this XML file, we get the output of:

```
username: ssencarnacion
total games: 2
CATAN : N/A
卡坦島 : 8.0
```

ASSUMPTION # 3: For comparison purposes, **we will be considering different versions of the game as the same game**. If a user has given different scores to different versions of the game, then we will take the highest score.

To implement this, we will no longer care about the game's name, but rather, its ID. If we encounter multiple ratings for a certain ID, then we will use the highest rating, if possible. We utilize a dictionary to keep track of the objectid.

```python
    # store highest rating per game ID
    ratings_by_id = {}

    for item in items:
        game_id = item.get("objectid")

        # skip items without objectid
        if not game_id:
            continue

        rating_value = item.find("stats/rating").get("value")

        try:
            rating_value = float(rating_value)
        except (TypeError, ValueError):
            rating_value = "N/A"

        # first time seeing this game ID
        if game_id not in ratings_by_id:
            ratings_by_id[game_id] = rating_value
        else:
            existing_rating = ratings_by_id[game_id]

            # numeric rating always beats N/A
            if existing_rating == "N/A" and rating_value != "N/A":
                ratings_by_id[game_id] = rating_value

            # higher numeric rating wins
            elif existing_rating != "N/A" and rating_value != "N/A":
                ratings_by_id[game_id] = max(existing_rating, rating_value)

    print(f"total unique games: {len(ratings_by_id)}")
    for game_id, rating in ratings_by_id.items():
        print(f"{game_id} : {rating}")
```

The output of parser_v2 on my XML file is the following.

```
username: ssencarnacion
total unique games: 1
13 : 8.0
```

b. parser_v2 Robustness

At this point in time, parser_v2 can only work on well-formed XML files, just like its predecessor. What happens if we try it on an erroneous XML file?

Running parser_v2 on the hive_engg XML file, we get the following output. It seems like we are in the clear for now, as this XML file is well-formed.

```
username: hive_engg
total unique games: 5
84876 : 9.0
13 : 7.0
110327 : 10.0
161936 : 9.0
167791 : N/A
```

We have the XML file of scarlettyellow. There is a username attribute inside the items tag (same as the case for hive_engg, actually). Moreover, the third item is missing a rating tag, which can be fatal for the current parser.

```
data > collection@stats=1&subtype=boardgame&username=scarlettyellow&version=0.xml
1  <?xml version="1.0" encoding="utf-8" standalone="yes"?>
2  <items totalitems="3" username="hidden_test_user">
3      <item objecttype="thing" objectid="0" subtype="boardgame">
4          <name sortindex="1">Resilience Test: Hex ID</name>
5          <stats minplayers="2" maxplayers="4">
6              <rating value="10">
7                  <average value="8.5" />
8              </rating>
9          </stats>
10     </item>
11
12     <item objecttype="thing" objectid="13" subtype="boardgame">
13         <name sortindex="1">Catan</name>
14         <stats>
15             <rating>
16                 <average value="2.0" />
17             </rating>
18         </stats>
19     </item>
20
21     <item objecttype="thing" objectid="84876" subtype="boardgame">
22         <name sortindex="5">The Castles of Burgundy</name>
23         <stats /> </item>        No rating tag
24 </items>
```

Upon running parser_v2 on scarlettyellow's XML file, we get the following result. This is the consequence of a missing rating tag. As seen in the error message, Python was trying to do None.get("value"), which is wrong.

```
25
26      rating_value = item.find("stats/rating").get("value")

Exception has occurred: AttributeError ×
'NoneType' object has no attribute 'get'
```

With defensive programming in mind, we can implement a simple fix for this case. The default rating value will be set to “N/A”. We will try to find a numerical rating, only if it is possible.

```python
        # safe extraction of rating value
        rating_value = "N/A"
        rating_elem = item.find("stats/rating")

        if rating_elem is not None:
            value = rating_elem.get("value")
            try:
                rating_value = float(value)
            except (TypeError, ValueError):
                pass
```

I’ve also adjusted the structure in the parsing logic to also look for the username in the items tag, if applicable. This will be prioritized over the username from the filename.

```python
    # parse XML
    root = ET.parse(filepath).getroot()
    items = root.findall("item")

    # override filename username if XML root provides one
    xml_username = root.get("username")
    if xml_username:
        username = xml_username

    print(f"username: {username}")
```

ASSUMPTION # 4: The username attribute in the items tag takes precedence over the username in the filename.

With this, our parser now works as intended. I’ve noticed that there is no actual game with an objectid of 0. But for our comparison purposes, we will be considering all possible values of a game’s objectid. Should it not be allowed, then we can simply add a check for it.

```
username: hidden_test_user
total unique games: 3
0 : 10.0
13 : N/A
84876 : N/A
```

ASSUMPTION # 5: Games can have a nonpositive objectid.

c. parser_v3 and Valid Usernames

At this point in time, our parser can now handle XML files with missing rating tag. But what about for cases like this one? We can see that the root tag is <errors>, instead of the usual <items>.

```
data > collection@stats=1&subtype=boardgame&username=someOneThatHopefullyWontExistPlsGuysDontCreateThisUser.xml
<?xml version="1.0" encoding="utf-8" standalone="yes" ?>
<errors>
    <error>
        <message>Invalid username specified</message>

    </error>
</errors>
```

Running parser_v3 on it, we get the following. It's good that our parser did not crash, but we have to account for cases where the root element is erroneous, in which we should disregard that XML file. Additionally, the username contains the .xml extension, which is the result of a faulty username extraction logic.

```
username: someOneThatHopefullyWontExistPlsGuysDontCreateThisUser.xml
total unique games: 0
```

ASSUMPTION # 6: XML files with their root element not set to <items> will not be processed.

To facilitate this assumption, we can add an early guard clause against XML files with <errors> as the root element.

```python
    # parse XML
    root = ET.parse(filepath).getroot()
    items = root.findall("item")

    # stop parsing if root element is <errors>
    if root.tag == "errors":
        print("Invalid username")
        return

```

```
Invalid username
```

As for the username extraction logic, I've utilized ChatGPT to implement a helper function. With this, we can now correctly extract the username from the filename.

```python
# simple helper to extract username from filename
# made with ChatGPT https://chatgpt.com/s/t_69699e7c20588191b0f27a93a9b44b4e
def extract_username(filepath):
    base = os.path.basename(filepath)
    return base.split("username=")[1].split("&")[0].split(".xml")[0]

def parser(filepath):
    # extract username from filename
    username = extract_username(filepath)
```

3.0 Bulletproofing the Parser

Now that we have improved our parser against the given sample XML files, it's time to further improve it. As much as possible, we want our parser to be resilient, since there might be XML files that are not *sanitized*.

a. Type Inconsistencies

From a given XML file, we are trying to parse information from it such as a username if it exists, the objectid of the games in their collection, and the corresponding rating for each one. Now, we will be doing operations on the ratings later.

I've utilized ChatGPT to create a sample XML file that would result to a type inconsistency. Observe that the ratings have different values.

parser_v4.py M collection@stats=1&subtype=boardgame&username=bad_actor.xml U

data > collection@stats=1&subtype=boardgame&username=bad_actor.xml

```xml
<?xml version="1.0" encoding="utf-8" standalone="yes"?>
<items totalitems="5" username="hive_engg" termsofuse="http://boardgamegeek.com/xmlapi/termsofuse">

  <!-- rating value should be integer, but it's text -->
  <item objecttype="thing" objectid="84876" subtype="boardgame">
    <name sortindex="5">The Castles of Burgundy</name>
    <stats minplayers="two" maxplayers="4">
      <rating value="Excellent">  Bad
        <average value="8.11543" />
      </rating>
    </stats>
  </item>

  <item objecttype="thing" objectid="13" subtype="boardgame">
    <name sortindex="1">Catan</name>
    <stats minplayers="3" maxplayers="4">
      <rating value="7">  Good!
        <average value="8.5" />
      </rating>
    </stats>
  </item>


  <!-- rating value is 'N/A' (non-numeric), average value is text -->
  <item objecttype="thing" objectid="167791" subtype="boardgame">
    <name sortindex="1">Terraforming Mars</name>
    <stats minplayers="1" maxplayers="5">
      <rating value="N/A">  Bad
        <average value="eight point four two" />
      </rating>
    </stats>
  </item>

</items>
```

I've improved the structure of the logic in the parser that handles cases like this. By default, we set a game's rating value to None by default. If it is possible, we then try to convert it to a float. If we encounter multiple ratings for a certain ID, then we will use the highest rating, if possible.

```python
        # attempt to parse rating, use None if missing/invalid
        rating_value = None
        rating_elem = item.find("stats/rating")

        if rating_elem is not None:
            try:
                rating_value = float(rating_elem.get("value"))
            except (TypeError, ValueError):
                pass  # leave as None if value is nonnumeric

        # store highest rating per game ID
        existing_rating = ratings_by_id.get(game_id)

        if existing_rating is None:
            ratings_by_id[game_id] = rating_value

        # update only if new rating is numeric and higher
        elif rating_value is not None and rating_value > existing_rating:
            ratings_by_id[game_id] = rating_value

```

Running the parser on the erroneous XML file, we get the following output. Note that the parser was already able to handle type inconsistencies like this. I only improved the structure to make it cleaner.

```
username: hive_engg
username: hive_engg
total unique games: 3
84876 : None
13 : 7.0
167791 : None
```

b. Unescaped Characters

The next thing we should be wary of is the presence of unescaped characters. According to https://www.w3.org/TR/xml/#charsets, the ampersand character **(&)** and left angle bracket **(<)** must not appear unescaped in character data or in attribute values.

> 2.4 Character Data and Markup
>
> Text consists of intermingled character data and markup. [Definition: **Markup** takes the form of start-tags, end-tags, empty-element tags, entity references, character references, comments, CDATA section delimiters, document type declarations, processing instructions, XML declarations, text declarations, and any white space that is at the top level of the document entity (that is, outside the document element and not inside any other markup).]
>
> [Definition: All text that is not markup constitutes the **character data** of the document.]
>
> The ampersand character (&) and the left angle bracket (<) MUST NOT appear in their literal form, except when used as markup delimiters, or within a comment, a processing instruction, or a CDATA section. If they are needed elsewhere, they MUST be escaped using either numeric character references or the strings " `&` " and " `<` " respectively. The right angle bracket (>) may be represented using the string " `>` ", and MUST, for compatibility, be escaped using either " `>` " or a character reference when it appears in the string " `]]>` " in content, when that string is not marking the end of a CDATA section.

The XML file looks well-structured, but there is the issue of unescaped characters.

parser_v4.py M collection@stats=1&subtype=boardgame&username=bad_actor.xml U

data > collection@stats=1&subtype=boardgame&username=bad_actor.xml

```xml
<?xml version="1.0" encoding="utf-8" standalone="yes"?>
<items totalitems="5" username="bad&actor" termsofuse="http://boardgamegeek.com/xmlapi/termsofuse">

  <item objecttype="thing" objectid="84876" subtype="boardgame">
    <name sortindex="5">The Castles of Burgundy & Friends</name>
    <stats minplayers="2" maxplayers="4">
      <rating value="9">
        <average value="8.11543" />
      </rating>
    </stats>
  </item>

  <item objecttype="thing" objectid="161936" subtype="boardgame">
    <name sortindex="1">Pandemic Legacy: Season 1 > "Special"</name>
    <stats minplayers="2" maxplayers="4">
      <rating value="9&">
        <average value="8.61245" />
      </rating>
    </stats>
  </item>

</items>
```

If we try to run our parser on this XML file, we get the following error:

```
25    root = ET.parse(filepath).getroot()

Exception has occurred: ParseError ×
not well-formed (invalid token): line 2, column 41
```

I utilized ChatGPT to make a function that automatically escapes *raw* ampersand characters. I will be operating under the following assumption:

> ASSUMPTION # 7: There will be no unescaped "<" characters. The only character that needs to be escaped are ampersands (&).

Before we actually do anything, we will try to sanitize the XML file and escape any raw ampersand characters.

```python
def escape_ampersands(xml):
    # Replace & with &amp; only if not part of an existing entity
    # made with ChatGPT https://chatgpt.com/s/t_6969e2d4d2188191b55f2b1c5c35d4f5
    return re.sub(r'&(?!amp;|lt;|gt;|quot;|apos;)', '&amp;', xml)

def parser(filepath):
    # extract username from filename
    # made with ChatGPT https://chatgpt.com/s/t_69699e7c20588191b0f27a93a9b44b4e
    username = os.path.basename(filepath).split("username=")[1].split("&")[0].split(".xml")[0]

    # read file and escape ampersands
    root = ET.fromstring(escape_ampersands(open(filepath, encoding="utf-8").read()))
```

With this, we can now successfully parse the XML file with unescaped ampersand characters.

```
username: bad&actor
total unique games: 2
84876 : 9.0
161936 : None
```

c. Catch-all Net

The main limitation of my parser at this point in time is that it will fail whenever there is an unclosed tag, resulting to a ParseError.

```
data > collection@stats=1&subtype=boardgame&username=bad_actor.xml
<?xml version="1.0" encoding="utf-8" standalone="yes"?>
<items totalitems="5" username="bad&actor" termsofuse="http://boardgamegeek.com/xmlapi/termsofuse">

  <item objecttype="thing" objectid="84876" subtype="boardgame">
    <name sortindex="5">The Castles of Burgundy &  Friends</name>
    <stats minplayers="2" maxplayers="4">
      <rating value="9">
        <average value="8.11543" />
      </rating>
    </stats>
  </item>

  <item objecttype="thing" objectid="161936" subtype="boardgame">
    <name sortindex="1">Pandemic Legacy: Season 1 > "Special"</name>
    <stats minplayers="2" maxplayers="4">
      <rating value="9&">
        <average value="8.61245" />
      </rating
    </stats>
  </item>

</items>
```

When we try to use the parser on this XML file, we will be greeted with the following error message:

```
23      # read file and escape ampersands
24      root = ET.fromstring(escape_ampersands(open(filepath, encoding="utf-8").read()))
```

```
Exception has occurred: ParseError ×
not well-formed (invalid token): line 19, column 4
```

To solve this problem, we will simply disregard malformed XML files. To facilitate that, we simply add a try/except block to guard against malformed XML files.

ASSUMPTION # 8: Malformed XML files will not be processed

```
23      # read file and escape ampersands
24      xml_content = open(filepath, encoding="utf-8").read()
25      xml_content = escape_ampersands(xml_content)
26
27      # catch-all: do nothing if XML is malformed
28      try:
29          root = ET.fromstring(xml_content)
30      except ET.ParseError:
31          print("Malformed XML")
32          return
```

We can now use our parser on the XML file.

```
' ;95ecab62-d7ce-4923-be77-dd3b56c2b5c0Malformed XML
```

3.1 Test Automation

We have covered all of our bases (hopefully). Now, we can try running our parser against various XML files. I've utilized ChatGPT to create a Python script that will run the parser function on all the relevant files in the data folder. Afterwards, the output will be collated in a log file for inspection.

```
test.py > ...
1   # made with ChatGPT https://chatgpt.com/s/t_6969ef9b644c81918f234226fd536d36
2   import os
3   import re
4   from parser_v6 import parser
5   import sys
6
7   DATA_FOLDER = "data"
8   LOG_FILE = "parser_log.txt"
9
10  # regex pattern to match the filename convention
11  FILENAME_PATTERN = re.compile(r"collection@stats=1&subtype=boardgame&username=.*\.xml")
12
13  def main():
14      # open log file in write mode
15      with open(LOG_FILE, "w", encoding="utf-8") as log:
16          # iterate through all files in the data folder
17          for filename in os.listdir(DATA_FOLDER):
18              if FILENAME_PATTERN.fullmatch(filename):
19                  filepath = os.path.join(DATA_FOLDER, filename)
20                  log.write(f"\nProcessing file: {filename}\n")
21                  log.write("-" * 40 + "\n")
22
23                  # temporarily redirect stdout to capture prints from parser
24                  old_stdout = sys.stdout
25                  sys.stdout = log
26                  try:
27                      parser(filepath)
28                  except Exception as e:
29                      print(f"Error processing {filename}: {e}")
30                  finally:
31                      sys.stdout = old_stdout
32
33                  log.write("-" * 40 + "\n")
34
35      print(f"Processing complete. Log written to {LOG_FILE}")
36
37  if __name__ == "__main__":
38      main()
39
```

Below is a snippet of the log.txt file. Our parser still works as intended!

```
Processing file: collection@stats=1&subtype=boardgame&username=bad_actor.xml
----------------------------------------
Malformed XML
----------------------------------------

Processing file: collection@stats=1&subtype=boardgame&username=hive_engg&version=0.xml
----------------------------------------
username: hive_engg
total unique games: 5
84876 : 9.0
13 : 7.0
110327 : 10.0
161936 : 9.0
167791 : None
----------------------------------------

Processing file: collection@stats=1&subtype=boardgame&username=scarlettyellow&version=0.xml
----------------------------------------
username: hidden_test_user
total unique games: 3
0 : 10.0
13 : None
84876 : None
----------------------------------------

Processing file: collection@stats=1&subtype=boardgame&username=someOneThatHopefullyWontExistPlsGuysDontCreateThisUser.xml
----------------------------------------
Invalid username
----------------------------------------

Processing file: collection@stats=1&subtype=boardgame&username=ssencarnacion.xml
----------------------------------------
username: ssencarnacion
total unique games: 1
13 : 8.0
----------------------------------------
```

4.0 Transformers

With our parser done, **we know that we can extract information properly**. The next stage here is to actually use that information so that we can do some comparisons among these users.

We'll modify our parser, shifting it away from printing statements to return a dictionary that is something like this. **We now need to transform our data**!

```
{
  "username": "stephen",
  "games": {
    "objectid1": 8.5,
    "objectid2": 9.0,
    "objectid3" : None
  }
}
```

With the help of ChatGPT, the parser function now returns a dictionary. Remember that ratings_by_id is a dictionary, so the underlying structure here will be a nested dictionary.

```python
    # create the JSON-like dict
    # made by ChatGPT https://chatgpt.com/s/t_6969f77652d48191bdf2528134dfbe40
    result = {
        "username": username,
        "games": ratings_by_id
    }

    return result

```

Again, with ChatGPT, we now have a script that runs the parser function on all the relevant files. We are working with **ASSUMPTION # 1**, in that all relevant files will follow a strict naming convention. This script will then save the returned dictionary as JSON objects, and save them in a new folder named parsed_data. Having the same username will overwrite the existing JSON file, so we'll operate with a new assumption.

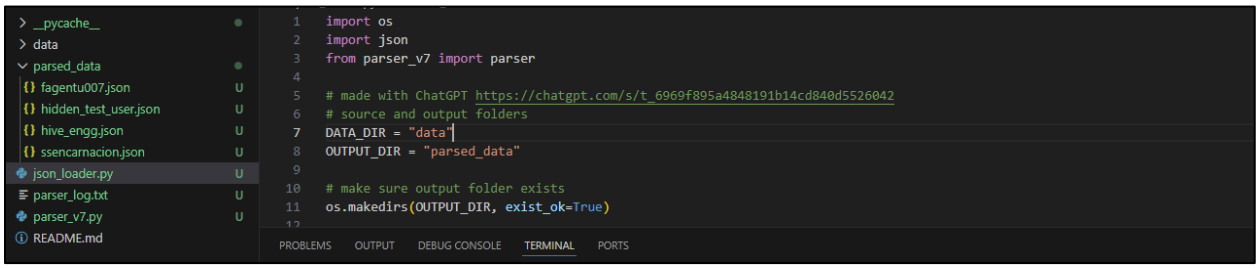
parser_v7.py U json_loader.py U

json_loader.py > filename

```python
import os
import json
from parser_v7 import parser

# made with ChatGPT https://chatgpt.com/s/t_6969f895a4848191b14cd840d5526042
# source and output folders
DATA_DIR = "data"
OUTPUT_DIR = "parsed_data"

# make sure output folder exists
os.makedirs(OUTPUT_DIR, exist_ok=True)

# define the filename pattern to match relevant files
# they all start with 'collection@stats=1&subtype=boardgame&username='
FILENAME_PREFIX = "collection@stats=1&subtype=boardgame&username="

# loop through files in data folder
for filename in os.listdir(DATA_DIR):
    if filename.startswith(FILENAME_PREFIX) and filename.endswith(".xml"):
        filepath = os.path.join(DATA_DIR, filename)

        # run parser
        result = parser(filepath)

        if result is not None:
            # save JSON output
            username = result["username"]
            output_path = os.path.join(OUTPUT_DIR, f"{username}.json")

            with open(output_path, "w", encoding="utf-8") as f:
                json.dump(result, f, indent=2, ensure_ascii=False)

            print(f"Parsed and saved: {username}.json")
        else:
            print(f"Skipped (malformed or errors root): {filename}")

```

ASSUMPTION # 9: Usernames are unique.

Upon running json_loader.py, we get the following result. **The resulting parsed_data files will only come from well-formed XML files**!



Here's what a sample JSON file looks like

5.0 Analysis Time

The question now is what metric/formula should we use to determine the similarity score of two users? To be honest, I don't even know where to begin. I would be relying on ChatGPT for this.

There is a variety of metrics that we can utilize for analysis: Cosine similarity, Pearson correlation, Euclidean Distance, Jaccard Similarity, etc. I asked ChatGPT which is best suited for the problem, and it recommended me Pearson Correlation.

After reviewing the problem, we decided to use the **Pearson correlation coefficient** as our primary metric. This choice is motivated by the fact that Board Game Geek users often have different rating habits — some users tend to rate consistently high, while others rate conservatively. Pearson correlation measures **the linear relationship between two users' ratings**, independent of their individual rating scales, making it ideal for detecting aligned tastes rather than absolute scores.

The **formula** for Pearson correlation between two users, U and V, is:

$$r = \frac{\sum_{i \in G}(R_{U,i} - \bar{R}_U)(R_{V,i} - \bar{R}_V)}{\sqrt{\sum_{i \in G}(R_{U,i} - \bar{R}_U)^2}\sqrt{\sum_{i \in G}(R_{V,i} - \bar{R}_V)^2}}$$

Where:

- G is the set of games that **both users have rated**.
- $R_{U,i}$ and $R_{V,i}$ are the ratings that users U and V gave to game i.
- $\bar{R}_U$ and $\bar{R}_V$ are the **average ratings** of users U and V across the games in G.

The result r ranges from -1 to 1:

- $r = 1$ → the users' tastes are perfectly aligned.
- $r = 0$ → there is no correlation between the users' ratings.
- $r = -1$ → the users' tastes are exactly opposite.

For display purposes, we normalize this to a **0–1 scale**:

$$\text{Similarity} = \frac{r+1}{2}$$

So a similarity of `1.0` indicates very similar tastes, while `0.0` indicates completely opposite tastes.

To further populate our parsed_data files, I had ChatGPT create some sample user data that would have varying similarities with user fagentu007.

parsed_data > {} boardgame_lover.json > ...

```
{
  "username": "boardgame_lover",
  "games": {
    "171623": 9.0,
    "34219": 6.0,
    "153938": 6.0,
    "102794": 9.5,
    "146021": 9.0,
    "154203": 7.0,
    "148949": 7.0,
    "54043": 6.0,
    "110327": 8.0,
    "134342": 8.0,
    "30549": 8.5,
    "121921": 9.0,
    "144733": 9.0
  }
}
```

very similar

parsed_data > {} moderate_player.json > ...

```
{
  "username": "moderate_player",
  "games": {
    "171623": 6.0,
    "34219": 6.5,
    "153938": 5.5,
    "102794": 7.0,
    "146021": 6.0,
    "154203": 6.0,
    "148949": 6.5,
    "54043": 6.0,
    "110327": 7.0
  }
}
```

moderately similar

parsed_data > {} casual_player.json > ...

```
{
  "username": "casual_player",
  "games": {
    "13": 7.0,
    "110327": 5.0,
    "84876": 6.0,
    "34219": 5.5,
    "102794": 6.0,
    "146021": 5.0,
    "154203": 5.0,
    "54043": 4.0,
    "121921": 6.0,
    "144733": 5.5
  }
}
```

moderately dissimilar

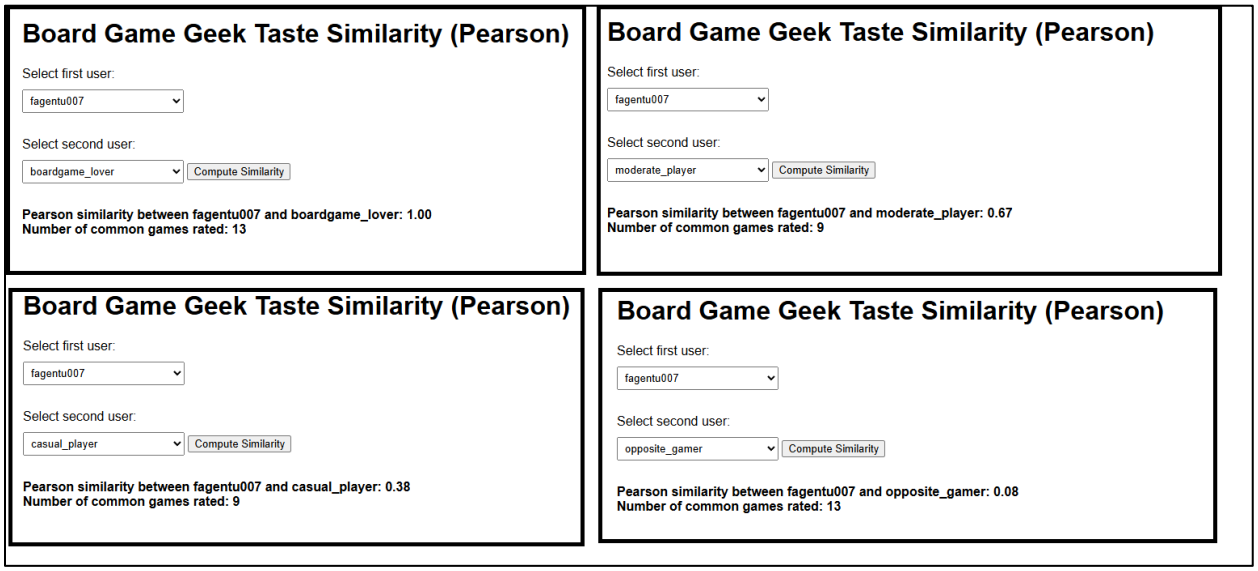
parsed_data > {} opposite_gamer.json > ...

```
{
  "username": "opposite_gamer",
  "games": {
    "171623": 3.0,
    "34219": 8.0,
    "153938": 8.0,
    "102794": 4.0,
    "146021": 3.5,
    "154203": 4.0,
    "148949": 5.0,
    "54043": 7.0,
    "110327": 3.0,
    "134342": 4.0,
    "30549": 3.5,
    "121921": 4.0,
    "144733": 3.5
  }
}
```

opposite tastes

We have all relevant files over at parsed_data folder. To test out this formula, let's have a simple webpage that allows us to pick two usernames. Afterwards, it will show us the similarity rating between these two users using the defined formula. For the nth time, I'll let ChatGPT handle it.

The webpage looks like this. It shows us the similarity between two users, and the number of common games they have rated.



Reflections/Recommendations

I would like to try and containerize the whole project into a docker container so that it can run anywhere without much hassle. I'm still in the process of learning about it, so I'll forgo it for now.

The data extraction process was made convenient due to my various assumptions, which almost felt like a cheat code. The parser could be made better if it can try to work on files without relying on a defined naming convention.

Given enough time, I would also want to try how to properly escape all *special* characters, rather than just escaping the ampersand (&). The process of catching any parsing error felt like a hack.

ChatGPT helped me out a lot in this project, especially for creating helper functions, running tests, and most importantly, in the data analysis phase. However, I would like to highlight that AI should not be used as a crutch, but rather, as a tool. It is only as good as the context you can provide it. I have a full understanding of the system design, and that's why I was able to easily conceptualize what kind of helper functions I needed, even as the system was evolving in nature. Without acquiring the right mental model of the system, you cannot hope for ChatGPT to save you. It's better to think of ChatGPT as something to rely on for repetitive tasks.

The process of selecting a proper formula / metric for defining similarity between users' taste in games was quite challenging for me, since I didn't really know where to start, as it can be considered quite *abstract*. This is where I think my reliance on ChatGPT can be considered dangerous, as I was letting it take the wheel for the analysis part.

Overall, this exercise was really refreshing for me, since I was able to create an actual data engineering project. Most of my previous activities / interviews were more of a coding round with DSA concepts, which I felt wasn't really indicative of one's ability as a developer. Here, challenged myself to think on how to implement a full system, given various tools at my disposal. I think the main challenge was in the extraction of data. There are so many things to consider, as we want to try and extract as much data we possibly can. Once I was able to do this successfully, I think the succeeding processes can now proceed smoothly. However, it seems that the data analysis part proved enough of a challenge on its own, because it's now up to you to make sense of the data that you have extracted.